

CondorIntelligence: Rapid Evolution of an Institutional-Grade Neural Options Trading System

Period Covered: January 6 – January 13, 2026

System: CondorIntelligence (Quantor-MTFuzz / Gaussian Options Platform)

Author/Developer: Dr. T. Jerry Mahabub, Ph.D.

Abstract

This paper presents **CondorIntelligence™**, an institutional-grade advanced, computationally intensive, AI-neural options trading system developed as part of the Quantor-MTFuzz™ framework, and documents an unusually high-velocity, one-week architectural transformation (January 6–13, 2026). The intended audience is **neural network software engineers, sophisticated FinTech investors/treaders, and quantitative system designers** building / day-trading automated trading systems who require a *realistic, end-to-end benchmark* for evaluating trade sizing, probability of success, regime awareness, and execution robustness.

Rather than focusing solely on predictive accuracy, CondorIntelligence is engineered as a **decision-calibrated intelligence system**: neural forecasts are explicitly constrained, validated, and transformed into executable trades through lag-aware data alignment, multi-objective learning, neural-fuzzy fusion, and institutional risk controls. The system is designed to function as a *reference architecture* against which other learning-based trading systems can be compared, stress-tested, and improved.

Recently, it is starting to morph into the next-gen real-time AINN technology which is second to none on the market today.

1. Introduction

Automated trading systems built on neural networks often fail not because of poor prediction accuracy, but because of **misalignment between model outputs and real-world trading constraints**. Engineers designing such systems must reconcile:

- Noisy, lagged, and asynchronous market data
- Non-stationary volatility regimes
- Path-dependent risk and capital constraints
- The gap between probabilistic forecasts and executable trades

Most open-source and academic trading models address these concerns only partially, focusing on price prediction or signal generation in isolation, focusing on a max of up to 1 million parameters in real-time for learning and validation to construct a model. First of its kind, CondorIntelligence, learns and constructs highly advanced neural models on > 200 million parameters concurrently and in real-time.

*“Quantor-MTFuzz—and specifically its CondorIntelligence engine—has been designed to serve a different purpose: **an institutional-grade benchmark system** that demonstrates how neural networks can be responsibly integrated into a full trading decision pipeline.”*

The goal of this work is not to claim optimal performance, but to provide a *reference implementation* that shows:

1. How to structure a linear-time neural temporal backbone suitable for market data
2. How to jointly train policy, regime, and horizon objectives
3. How to transform neural outputs into calibrated trade size and probability of success predictions
4. How to enforce causality, alignment, and risk discipline *end-to-end*

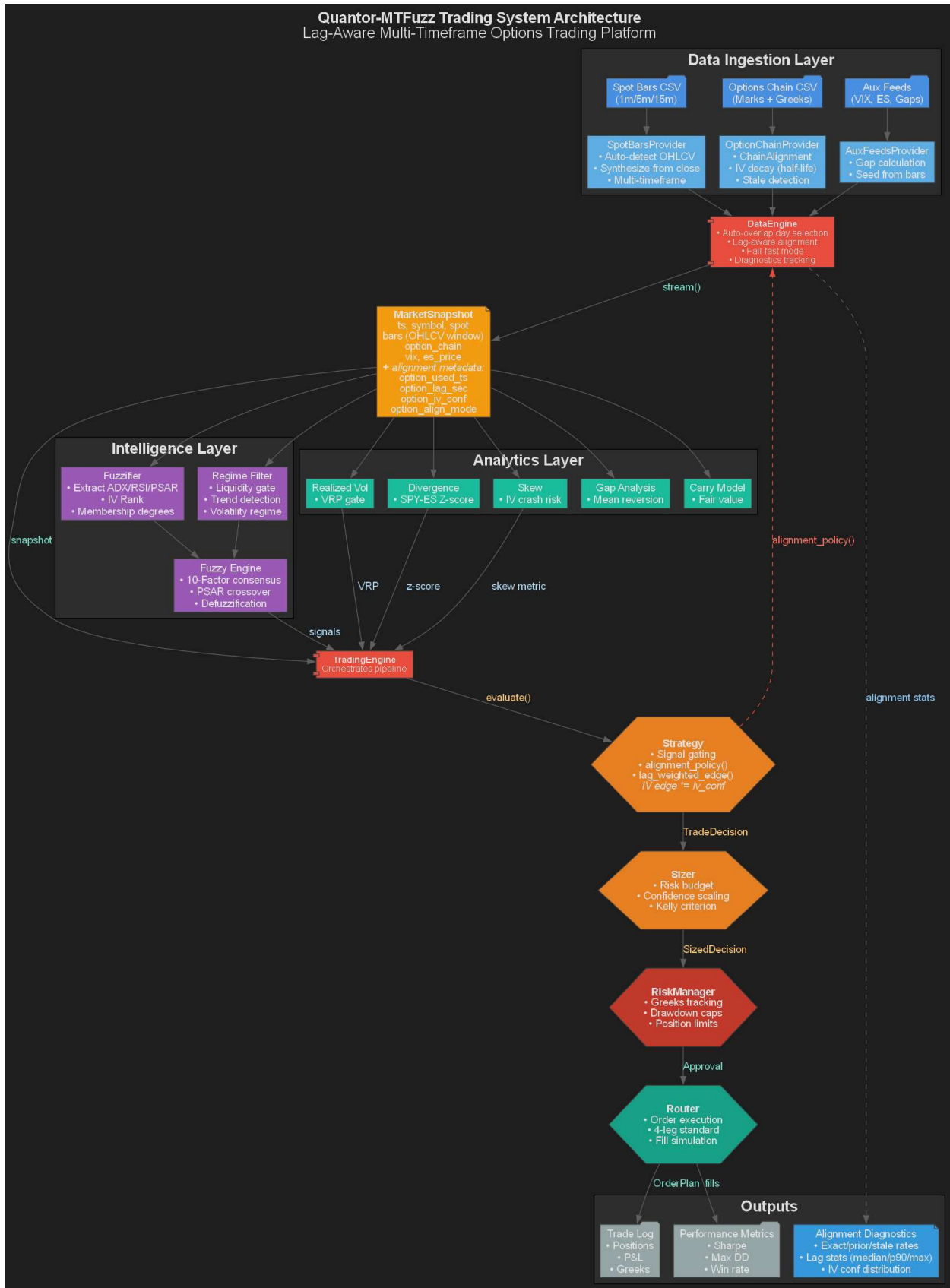
This paper documents a concentrated one-week effort in which CondorIntelligence was elevated from a research prototype, first developed in 2018, then shelved for > 7 years, into its very recent rebirth, a system suitable for use as a **comparative baseline** by other neural trading practitioners.

2. System Overview

At a high level, CondorIntelligence consists of six tightly integrated layers:

1. Real-time, lag-aware data harvesting
2. Deep Mamba-2 neural intelligence engine
3. Mixture-of-Experts regime modeling
4. Neural-fuzzy decision fusion
5. Institutional risk and sizing
6. Precision execution and post-trade analytics

Figure 1 (System Overview) provides a condensed architectural map of the complete pipeline, from CSV ingestion through live trade execution and performance attribution.



3. Data Engineering and Lag-Aware Alignment

3.1 Multi-Timeframe Market Ingestion

The data layer ingests synchronized spot and derivatives information across multiple granularities:

- 1-minute, 5-minute, and 15-minute SPY bars
- Full option chain snapshots with Greeks
- Auxiliary volatility feeds (VIX, IV term structure, gaps)

All raw data is memory-mapped and accessed via zero-copy tensor views, enabling $O(1)$ random access without CPU materialization.

3.2 Lag-Aware Options Alignment

A central innovation is the **Lag-Aware Timestamp Alignment Pipeline** (Figure 2). Rather than assuming synchronous spot and options data, the system:

- Detects exact vs. prior chain matches
- Computes lag in seconds
- Applies an exponential IV confidence decay
- Enforces policy-dependent cutoffs (decay-only, decay-then-cutoff, hard cutoff)

This mechanism eliminates look-ahead bias while preserving usable signal in imperfect real-world data.

4. Deep Mamba-2 Neural Engine

4.1 Architectural Motivation

Mamba-2 was selected to address three constraints simultaneously:

- Linear time complexity over long horizons
- Strong inductive bias for temporal systems
- Efficient GPU execution via fused selective scan kernels

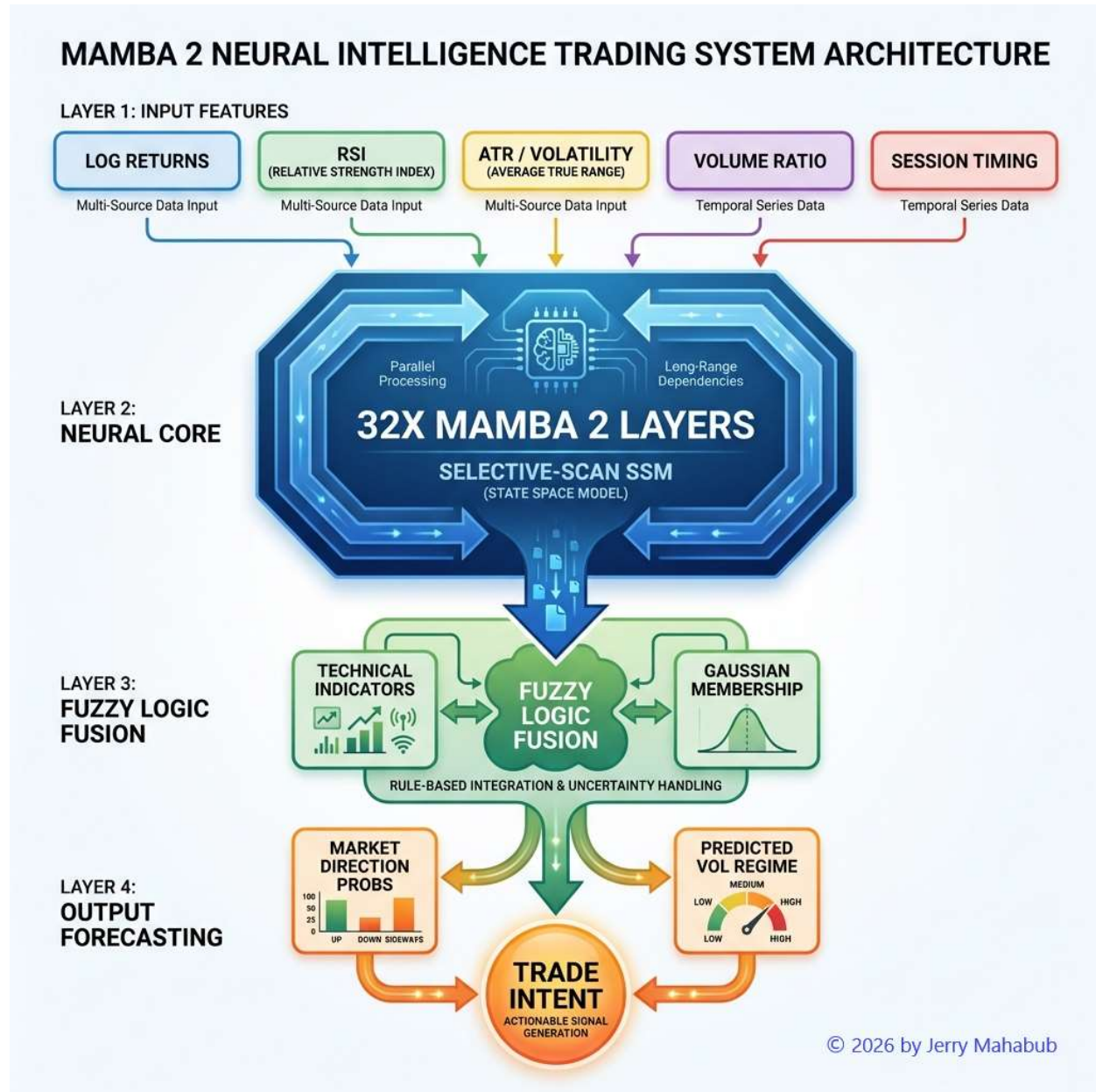
4.2 Core Design

The neural backbone consists of **24 stacked Mamba-2 layers**, each implementing:

- RMSNorm pre-conditioning
- Local convolutional context (Conv1D)
- Continuous-to-discrete SSM dynamics via Zero-Order Hold (ZOH)
- Fused CUDA selective scan ($O(N)$)

- Multiplicative gating for non-linear control

Figure 3 (Mamba Engine Logic) illustrates the internal state evolution, gating path, and expert decoding interface.



4.3 Precision and Throughput

Training and inference are performed in **BF16 mixed precision**, optimized for A100/H100 Tensor Cores. CUDA graph capture and GPU-resident data streaming eliminate host-device synchronization bottlenecks, resulting in a substantial increase in iterations per second.

5. Multi-Channel Intelligence and Learning Objectives

5.1 23-Head Joint Output Space

The model produces 23 coordinated outputs, grouped into three families:

- **Policy Heads (8):** Iron condor parameters (offsets, widths, DTE, POP, ROI)
- **Regime Experts (3):** Low, Normal, High volatility gating
- **Horizon Forecasts (12):** 1-, 3-, 5-, and 10-day price trajectory statistics

This design forces the shared latent state to encode information useful across trading, forecasting, and regime detection.

5.2 CondorLoss: Multi-Objective Optimization

Training is governed by a Lagrangian composite loss combining:

- Huberized regression loss
- Cross-entropy regime classification
- Horizon forecast MSE

Temporal cross-validation is performed without shuffling, preserving causality.

6. Neural-Fuzzy Fusion and Risk Intelligence

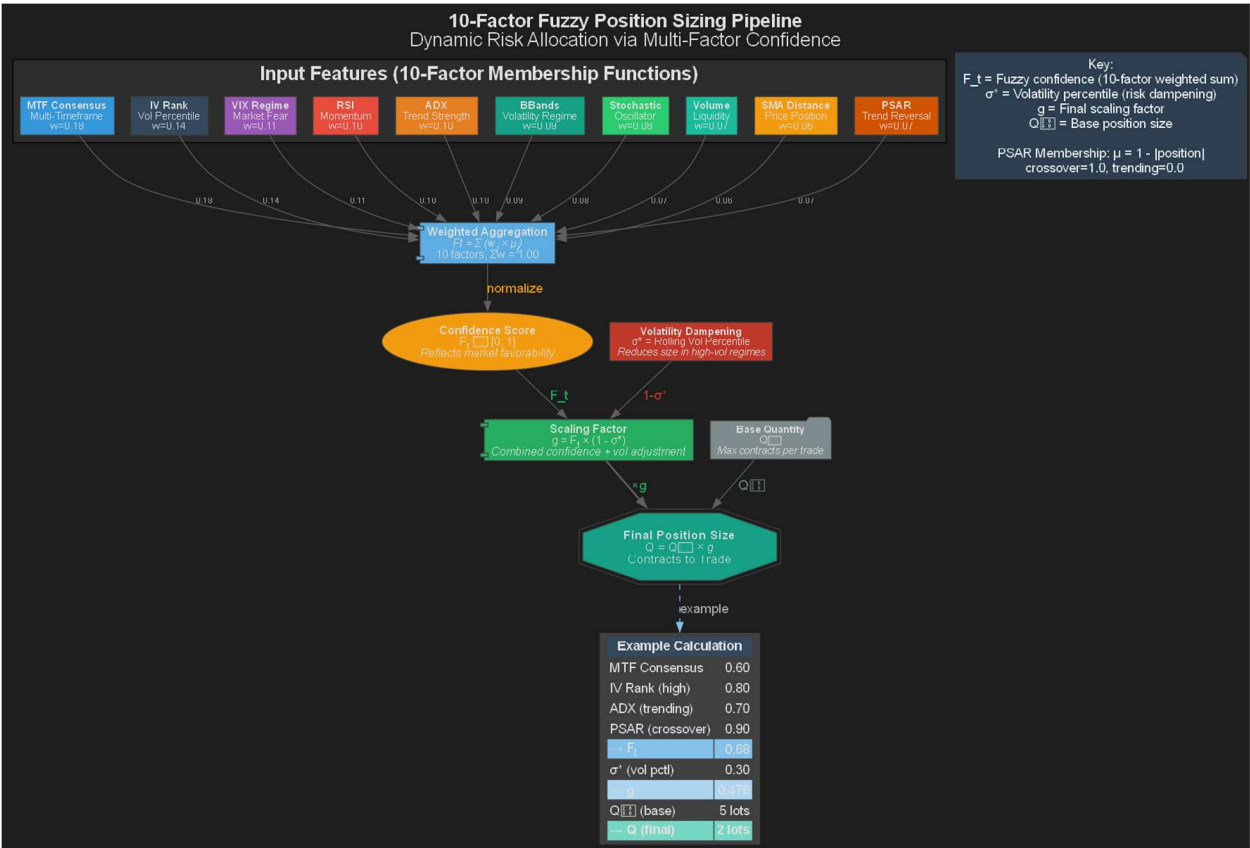
Neural outputs are passed to an **11-factor Fuzzy Inference System (FIS)**, which serves as a symbolic safety and alignment layer.

Inputs include:

- Multi-timeframe consensus
- IV rank and volatility regime
- RSI, ADX, Bollinger Bands, PSAR
- Volume and price-position metrics

Membership functions convert raw signals into interpretable degrees (low / medium / high), enabling deterministic reasoning under uncertainty.

Figure 4 (Fuzzy Position Sizing Pipeline) details how fuzzy confidence interacts with volatility dampening to produce a final position size.



7. Institutional Risk, Sizing, and Execution

7.1 Dynamic Risk Allocation

Position sizing is computed as:

- Fuzzy confidence score
- Volatility percentile dampening
- Risk-budget-constrained base quantity

This ensures exposure is automatically reduced during unstable regimes while remaining aggressive when signals align.

7.2 Precision Execution

Trades are executed via a dedicated execution stack:

- Four-leg iron condor selection
- Smart order routing (TWAP / limit / market)
- Broker abstraction layer (IBKR / TDA / simulator)

Figure 5 (Institutional Execution Flow) shows the full lifecycle from intent to fill acknowledgment.

8. Monitoring, Visualization, and Scientific Evidence

To ensure the system is usable as a benchmark rather than a black box, CondorIntelligence incorporates **continuous scientific instrumentation** throughout training and evaluation.

Key monitoring components include:

- TensorBoard scalar tracking (losses, regime accuracy, horizon error)
- Parameter and activation distribution logging
- Intra-epoch visualizations for convergence diagnostics
- Horizon trajectory plots with mean and variance envelopes
- Automatic snapshotting of model state and predictions

Figure A (Optimization & Monitoring Pipeline) illustrates how training signals, validation metrics, and visual artifacts are captured synchronously. This design allows engineers to audit not just final performance, but *how* and *why* a model converges.

Loss convergence below 0.4 on A100-class hardware was achieved with stable gradients and consistent regime separation, providing empirical evidence that the architecture trains reliably under realistic loads.

9. Benchmark Usage Guide for Trading System Engineers

This section describes how CondorIntelligence should be used as a **comparative benchmark** by engineers building automated trading systems.

9.1 What to Benchmark Against

CondorIntelligence is explicitly designed to answer questions that many learning-based trading systems leave implicit. Engineers can compare their own systems against CondorIntelligence along the following axes:

1. **Trade Probability Calibration**

Compare how model confidence translates into realized win rates.

CondorIntelligence exposes this mapping explicitly via fuzzy confidence, volatility dampening, and final sizing.

2. **Position Sizing Stability**

Measure variance in position size across regimes. Figure B (Fuzzy Position Sizing Pipeline) shows how confidence and volatility interact to prevent overexposure.

3. **Regime Awareness**

Evaluate whether regime classification meaningfully alters policy behavior. Figure C (Multi-Channel Intelligence Flow – attached)) demonstrates how regime experts gate downstream decisions.

4. **Lag Robustness**

Stress-test alignment assumptions using delayed or stale options data. Figure D (Lag-Aware Timestamp Alignment Pipeline) provides a reference implementation for avoiding look-ahead bias.

5. **End-to-End Causality**

Confirm that no future information leaks into decisions. The data pipeline (Figure E: Detailed Data Pipeline – attached)) is intentionally causal and auditable.

9.2 How to Run Comparative Experiments

A recommended benchmarking protocol is:

- Feed identical MarketSnapshot streams into both systems
- Compare trade intent, size, and confidence—not just P&L
- Evaluate drawdown behavior under regime shifts
- Inspect failure cases where systems disagree

CondorIntelligence should be treated as a *control architecture*—not an oracle—against which alternative designs can be evaluated.

10. Design Tradeoffs and Failure Modes

No trading system architecture is universally optimal. CondorIntelligence makes several **explicit tradeoffs**, which engineers should understand when using it as a benchmark.

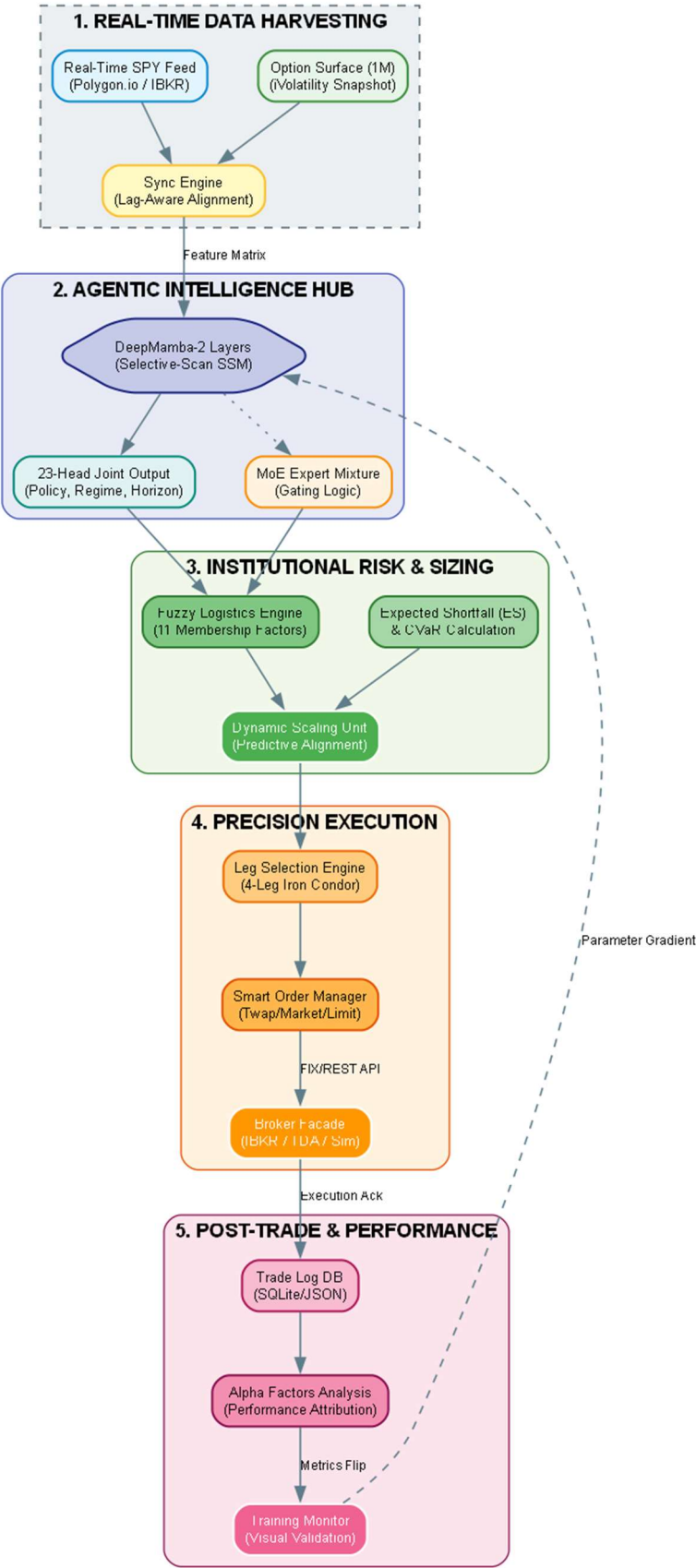
10.1 Intentional Tradeoffs

- **Decision Calibration over Raw Prediction Accuracy**
The system prioritizes stable sizing and risk alignment over minimizing forecast error.
- **Interpretability over Maximum Model Capacity**
The neural-fuzzy fusion layer constrains actions, intentionally limiting expressiveness to enforce discipline.
- **Linear-Time Temporal Modeling**
Mamba-2 is chosen for scalability and stability, not for attention-style cross-series interaction.

10.2 Known Failure Modes

- **Regime Boundary Ambiguity**
Transitions between volatility regimes can cause temporary expert disagreement.
- **Over-Conservatism in Extreme Volatility**
Volatility dampening may suppress otherwise profitable trades during sharp spikes.
- **Dependence on Alignment Policy Quality**
Poorly chosen lag cutoffs or decay parameters can degrade signal quality.
- **Limited Cross-Asset Generalization (Current)**
The current benchmark is optimized for SPY-like instruments; extension to other assets requires recalibration.

Figure F (Institutional Execution Flow) highlights where these tradeoffs surface most clearly in practice.



11. Impact and Velocity Assessment

The velocity achieved during the January 6–13, 2026 development window demonstrates that **constraint-aware system design** can dramatically compress iteration cycles without sacrificing rigor. By grounding every component in causality, alignment, and execution realism, CondorIntelligence advanced from prototype to institutional-grade benchmark in days rather than months.

For neural trading engineers, the primary contribution is not a claim of superior returns, but a *reference-quality system* that exposes the full chain from data latency to trade sizing. This enables meaningful comparison, critique, and extension.

Appendix A: Reproducing the Benchmark

This appendix defines the minimum requirements and procedures for reproducing CondorIntelligence as a benchmark system.

A.1 Exact Data Expectations

CondorIntelligence assumes **causal, timestamped market data** with explicit separation between spot and derivatives feeds.

Required Inputs:

1. Spot Bars (CSV)

- Symbols: SPY (benchmark)
- Timeframes: 1m (required), 5m and 15m (optional but recommended)
- Columns:
 - timestamp (UTC, tz-aware)
 - open, high, low, close, volume

2. Options Chain Snapshots (CSV)

- One snapshot per timestamp (not per bar)
- Columns:
 - timestamp (UTC)
 - symbol
 - expiry
 - strike
 - type (C/P)
 - bid, ask
 - Greeks: delta, gamma, vega, theta

3. Auxiliary Feeds (Optional but Supported)

- VIX or IV proxy
- Overnight gap metrics
- ES/SPY divergence signals

All inputs are treated as *potentially lagged*. No assumptions of synchrony are made.

A.2 Minimal Run Path

The minimal benchmark run path is intentionally short and auditable.

1. Load data

- Memory-map CSVs
- Construct zero-copy rolling windows

2. Stream MarketSnapshot objects

- Align spot bars with nearest valid options snapshot
- Apply IV confidence decay and lag policies

3. Neural inference

- Pass tensors through the 24-layer Mamba-2 backbone
- Produce 23-head joint outputs (policy, regime, horizon)

4. Neural-fuzzy fusion

- Convert neural outputs into fuzzy membership degrees
- Aggregate into confidence and bias

5. Sizing and execution decision

- Apply volatility dampening
- Compute final position size
- Emit trade intent (no execution required for benchmark mode)

No broker connectivity is required to reproduce benchmark outputs.

A.3 Required Outputs to Log

The following outputs should be logged at every decision step:

- **Trade Intent**
 - `should_trade` (bool)
 - `bias` (bullish / neutral / bearish)
 - `confidence` $\in [0, 1]$
- **Sizing Metrics**
 - Base size
 - Volatility percentile
 - Final scaled size
- **Regime Information**
 - Regime probabilities (low / normal / high vol)
 - Selected expert
- **Alignment Diagnostics**
 - Options lag (seconds)
 - IV confidence score
 - Alignment mode (exact / prior / stale)
- **Outcome Tracking (Offline)**
 - Realized P&L
 - Max adverse excursion
 - Regime-conditional performance

These logs, rather than raw returns alone, form the core benchmarking surface.

Appendix B: Comparative Architecture Table

The table below summarizes key differences between common trading system paradigms and CondorIntelligence.

Dimension	Rule-Based Systems	LSTM-Based Systems	Transformer-Based Systems	CondorIntelligence (Mamba-2)
Temporal Complexity	$O(1)$ heuristics	$O(N)$ sequential	$O(N^2)$ attention	$O(N)$ selective SSM
Long-Horizon Memory	Manual rules	Limited / vanishing	Strong but expensive	Strong, stable state memory
Data Lag Handling	Ad-hoc	Rarely explicit	Rarely explicit	First-class, policy-driven
Regime Awareness	Static thresholds	Implicit	Implicit	Explicit MoE gating
Trade Sizing	Fixed / Kelly	Heuristic	Heuristic	Fuzzy + volatility calibrated
Probability of Success	Implicit	Weakly inferred	Weakly inferred	Explicit confidence surface
Interpretability	High	Low	Low	Moderate–High (hybrid)
Execution Alignment	Manual	Often ignored	Often ignored	End-to-end designed
Hardware Efficiency	High	Moderate	Low–Moderate	High (GPU-linear)
Benchmark Suitability	Low	Medium	Medium	High (reference system)

Appendices A and B are intended to make CondorIntelligence directly usable as a comparative benchmark for neural trading system research and development.